Agile Agentic ASPICE (AAA)

From "Vibe Coding" to an Autonomous, Control-Loop-Governed AI Software Factory

Scrum Sprints · 5 ASPICE Stages · Independent Quality Gates · 3-Loop Cascade Control

1. Process Guardrails

5 coupled ASPICE stages with independent **Quality Gates** prevent shortcuts, scope creep, and untested code.

2. Cascade Control

Fast inner feedback loops (compiler, tests, gatekeeper agent2) reject hallucinations and drift **before** human review.

3. Time Leverage

Only **2–3h of focused human time** per sprint instead of constant 15-minute wait loops. Agents work **up to 24h/day**.

Rainer Blind | Architecture & Process Blueprint (`aTrainingTracker`)

October 20, 2026

The Core Idea: Why AAA (*Agile Agentic ASPICE*)?

1. Agile (Scrum Cadence)

- **Iterative Sprints** with strict backlog prioritization (*Rank Discipline*).
- **Ceremony 1 (Start):** Joint ticket screening (Human + Agent).
- **Strategy A:** Shared `sprint/<ID>` branch prevents merge hell.
- **Ceremony 2 (End):** Live on-device review & *Retro Before Merge*.

2. Agentic (Separation of Powers)

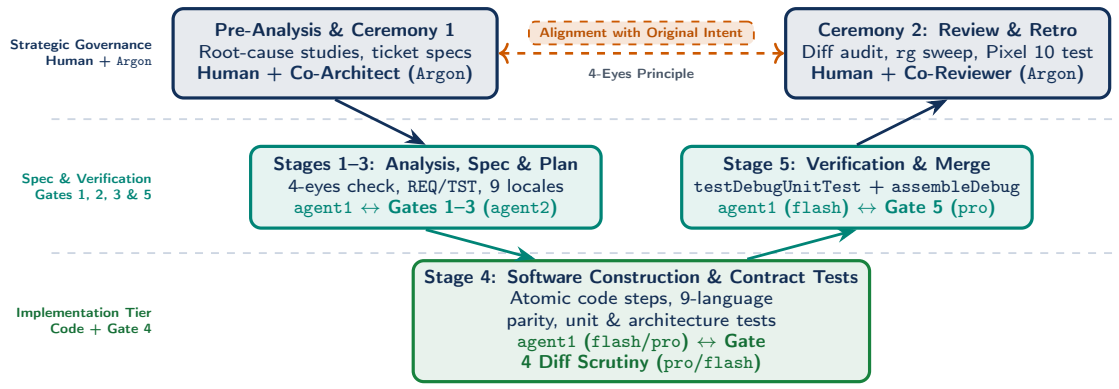
- **Product Owner (Human):** Vision, priorities & final UX sign-off.
- **Co-Architect (`Argon`):** Root-cause studies, Ceremony 1 & 2 audit.
- **Execution Subagents (`agent1`):** `pro/flash` across Stages 1–5.
- **Gatekeeper (`agent2`):** Independent `research` subagent (*Zipper*).

3. ASPICE (Speed Bumps)

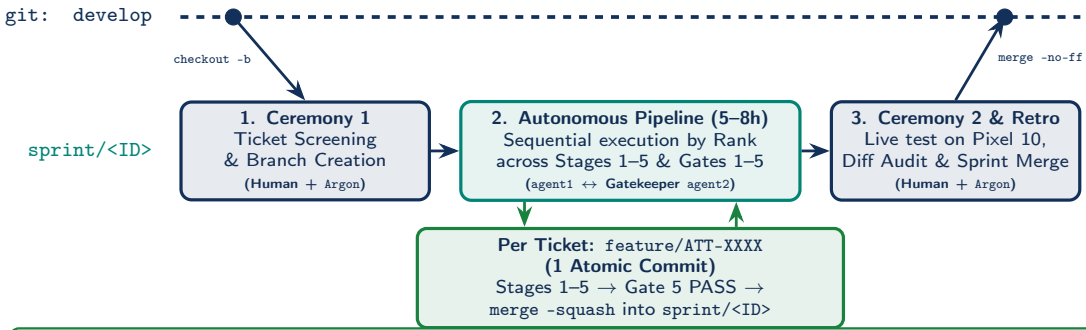
- **Deliberate "Speed Bumps":** No code before analysis, spec & plan.
- **5 Coupled Sub-Tasks** governed by strict Quality Gates (Gates 1–5).
- **Chesterton's Fence:** Mandatory proof before altering existing REQs.
- **8 Core Rules & Living Docs:** Compact (~5 KB) permanent memory.

Key Insight from Automotive Engineering: In the automotive industry, A-SPICE introduces deliberate process guardrails so engineers cannot cut corners on safety-critical software. For AI agents operating at 100× speed—which naturally gravitate toward the path of least resistance—these **speed bumps are the essential safety net**.

Workflow 1: The Hybrid V-Model (Human & Argon ↔ Autonomous Pipeline)



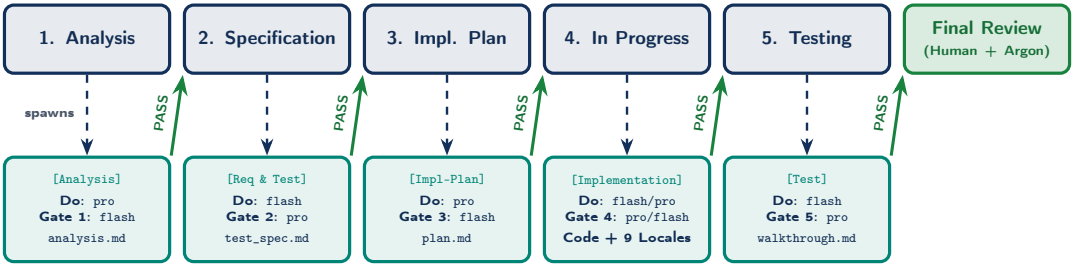
Workflow 2: The Scrum Sprint & Git Branching Flow (Strategy A)



Why Strategy A (`sprint/<ID>`) Eliminates Merge Hell: Every ticket is squashed into a **single atomic commit** on the shared `sprint/<ID>` branch immediately after **Gate 5 PASS**. Ticket $k + 1$ starts on top of the verified code of ticket k—preventing diverging branches at sprint end.

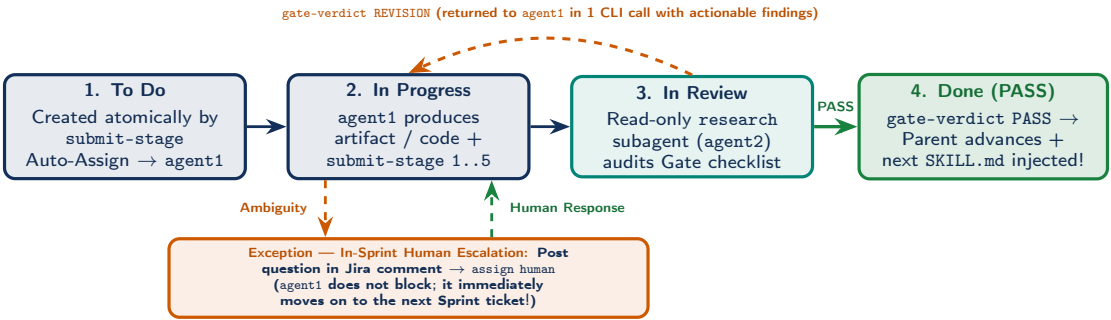
Workflow 3: The 2-Tier Cascade & Asymmetric pro/flash Zipper Principle

Tier 1: Parent Ticket Status in Jira (synchronously advances via `gate-verdict PASS` + auto-injects next `SKILL.md`)

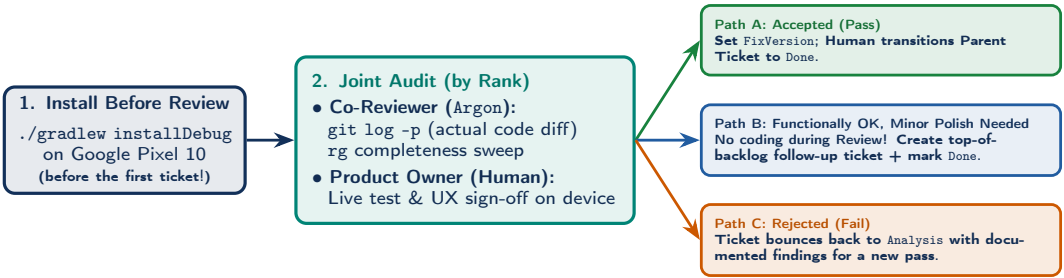


Tier 2: Asymmetric `pro/flash` Zipper Principle (`submit-stage` → read-only `research` subagent → `gate-verdict`)

Workflow 4: The Sub-Task State Machine & Single-Turn Composite CLI



Workflow 5: Ceremony 2 — Joint Review & Decision Tree



After the Final Ticket — Mandatory Retrospective (*Retro Before Merge*): Before merging `sprint/<ID>` into `develop`, Human and `Argon` jointly analyze: Which issues slipped past the automated checks and quality gates (Gates 1–5)—and how do we sharpen our Document Triad and skills for the next sprint?

Detailed Gate Criteria: Stage 1 (Gate 1) & Stage 2 (Gate 2)

Stage 1: [Analysis] (pro) → Gate 1 (flash)

Artifact: analysis/<KEY>_analysis.md | Analysis → Test Spec

Role of agent1 (pro — 4-Eyes Principle): Validates the ticket pre-analysis as a hypothesis against the actual code on sprint/<ID>; identifies all call sites, existing REQ IDs, and system invariants.

Gate 1 Checklist (agent2 research Subagent — flash):

1. **Root Cause vs. Symptom:** True cause proven in code?
2. **User Scope Grounding:** No unrequested rewrites?
3. **Blast Radius:** All call sites & affected modules listed?
4. **System Invariants:** Stated what *must not* break?
5. **Risk Classification:** LOW / MEDIUM / HIGH justified?

Stage 2: [Req & Test Spec] (flash) → Gate 2 (pro)

Artifact: test_specs/<KEY>_test_spec.md | Test Spec → Impl. Plan

Role of agent1 (flash — Living Docs & 9 Locales): Updates requirements.md (REQ-XXX), tests.md (TST-XXX), defines *Given-When-Then* criteria, and plans string resources across all 9 languages.

Gate 2 Checklist (agent2 research Subagent — pro):

1. **Atomic Syntax:** SHALL/MUST without implementation leaks?
2. **Bidirectional Traceability:** Every REQ-XXX ↔ TST-XXX?
3. **Chesterton's Fence (4-Field Mandate):** Whenever existing REQs are modified or removed, *Original ID*, *Git Commit/Ticket*, *Historical Reason*, and *Invariant Preservation* must be proven—otherwise REVISION!

Detailed Gate Criteria: Stages 3, 4 & 5 (Gates 3, 4 & 5)

Stage 3 (pro) → Gate 3 (flash)

Sub-Task: [Impl-Plan]
Artifact: plans/<KEY>_plan.md

- **SWE.2 Architecture:** Layer & component boundaries respected?
- **DB Schema Safety:** Parity between onCreate and onUpgrade?
- **Test Mapping:** Every atomic step mapped to a test in tests.md?
- **Code Lock:** Enforced via jira_util.py check-gate!

Stage 4 (flash/pro) → Gate 4

Sub-Task: [Implementation]
Focus: Code, Tests & 9 Locales

- **Diff Scrutiny:** Does actual git diff match the Stage 3 plan?
- **No Scope Creep:** No unrelated files or unauthorized refactoring?
- **Contract Tests:** Targeted unit tests green in 5–15s?
- **9-Locale Parity:** All 9 language files (EN..PT) completely updated?

Stage 5 (flash) → Gate 5 (pro)

Sub-Task: [Test]
Artifact: walkthrough.md

- **Unit Tests:** testDebugUnitTest 100% green (0 failures)?
- **Resource Build:** assembleDebug (AAPT2) 100% error-free?
- **Living Docs:** requirements.md & tests.md set to Verified?
- **1 Atomic Commit:** Squashed onto sprint/<ID>?

Why Jira Cloud as the External State Machine & Telemetry Backbone?

1. Immutable Out-of-Context State Machine & Audit Trail

- Zero State Loss Across Compactions: **Survives LLM context compactions, session restarts, and subagent handoffs with 100% deterministic state.**
- Role-Segregated Identity: **Enforces distinct accounts (**`agent1`, `agent2`, `coordinator`, `human`**), blocking parent** `Done` **transitions and timestamping every stage deliverable and gate verdict.**

2. Empirical Process Telemetry (35 Sprints / 423 Tickets)

- Quantitative Bottleneck Discovery: **Mining $>$ 2,100 sub-task changelogs measured** `S1-S5` **durations,** `pro` **vs.** `flash` **defect rates, and test parallelization (**`S5` **cut from 14.5 min to 4.0 min).**
- Breaking the "40-Turn Wall": **Revealed that ~70% of ticket time was sequential Jira CLI turns—motivating composite** `submit-stage` **&** `gate-verdict` **commands.**

3. Asynchronous Decoupling of Human & Autonomous Pipelin

- Concurrent Non-Blocking Collaboration: **While the autonomous pipeline executes the active sprint in the background, the human can file new tickets, refine the backlog, answer escalated sub-task questions (**`Rule 6`**), and queue retrospective improvements (**`ATT-4290`**) without interrupting the running agent.**

4. Deterministic Active Context Injection (`jira_util.py`)

- Zero-Turn Skill Reloading: **Stage transitions in Jira (**`gate-verdict PASS`**) act as the deterministic trigger to print the exact next stage's** `SKILL.md` **and the 8 Core Governance Rules (~5 KB) directly to** `stdout`**—preventing "Lost in the Middle" context drift with 0 extra LLM turns.**

Infinite Agent Context & Why We Do Not Allow a "Fast-Track"

1. Artifacts as "Infinite Agent Context"

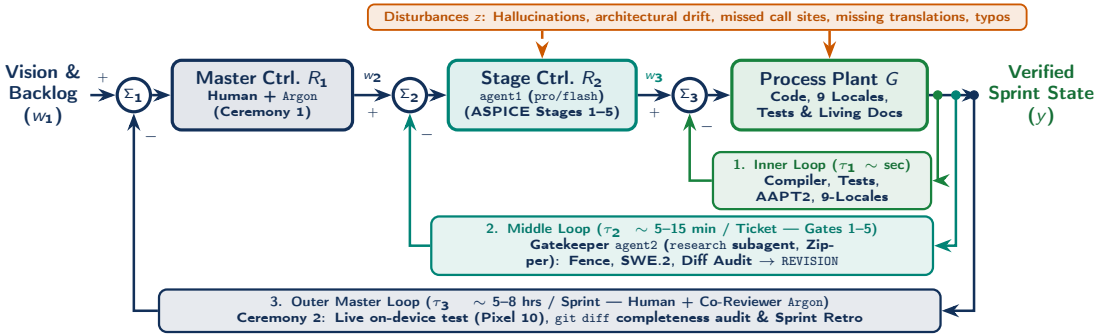
- **The Problem of Ephemeral Chats:** An LLM context window ends with the session—implicit knowledge discussed only in chat is lost forever.
- **The Repository as Long-Term Memory:** Through `docs/engineering/`, `requirements.md`, `tests.md`, and the **Document Triad**, a permanent knowledge base lives directly in Git.
- **Real-World Leverage:** When analyzing new bugs or features, agents automatically read prior walkthrough and spec documents, immediately grasping the historical *"Why"*.

2. Why There Is No "Fast-Track"

1. **Slippery-Slope Effect:** Once a "Fast-Track" for supposedly trivial tickets exists, convenience ensures it gets used every time—eroding process discipline immediately.
2. **Zero Agent-Initiated Shortcuts:** Agents must never propose skipping stages or quality gates on their own initiative.
3. **Mandatory Pushback:** If the human requests an exception, the agent *must* first object with a reasoned risk assessment (documented deviation).

Result: Maximum autonomous development velocity (24/7) with full automotive-grade traceability and institutional memory.

Control Engineering Principle: Agile Agentic ASPICE as Cascade Control



Why Cascade Control ($\tau_1 \ll \tau_2 \ll \tau_3$)? Disturbance Rejection Before Reaching the Human! Fast inner loops (τ_1: compiler/tests, τ_2: independent gatekeeper agent2) reject disturbances (z) right at the source **before** they propagate to the slow outer master loop (τ_3). That is why the human does not need to intervene every 10 minutes!

Time Allocation: Vibe Coding vs. Agile Agentic ASPICE

A. Classic "Vibe Coding": Human Trapped in a 15-Minute Wait Loop (Active Only a Few Hours/Day)

Human:	1. Prompt	Waits	2. Review	Waits	3. Prompt	Waits	4. Review	**18–20 Hours / Day Standstill** (Zero progress whenever human leaves desk)
Agent:	Idle	5–20 min	Idle	5–20 min	Idle	5–20 min	Idle	

B. Agile Agentic ASPICE: 2–3h Focused Human Time per Sprint — Up to 24h/Day Autonomous Agent Execution

Human (2–3h):	Ceremony 1 (∼1h) Ticket Screening	5–8 Hours Completely Free (Day Job, Sports, Sleep) or asynchronous brainstorming in Chat 2	Ceremony 2 (∼1.5h) Review & Retro	Free / Overnight (Sprint $N + 1$ runs)
Chat 1 (Sprint):	Co-Architect (`Argon`)	5–8h Autonomous Execution (Stages 1–5, Gates 1–5) `agent1` ↔ `agent2` (Code, 9 Locales, Tests, Docs)	Co-Reviewer (`Argon`)	Sprint $N + 1$ (5–8h) Up to 24h/Day Active
Chat 2 (Backlog):		Parallel in Chat 2: Brainstorming, Studies & Backlog (prepares Sprint $N + 1$ without disturbing Sprint N)		

The Paradigm Shift: In **Vibe Coding**, the human is reduced to a clocked "agent babysitter". In **Agile Agentic ASPICE**, the human invests only **2–3 highly focused hours per sprint** (screening at start, review & retro at end)—while agents work autonomously for **5–8 hours per sprint** (up to **24h/day**).